

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC PHENIKAA**



BÁO CÁO BÀI TẬP LỚN

**HỆ THỐNG GỢI Ý SẢN PHẨM HAI GIAI ĐOẠN
TÍCH HỢP TRIỆU HỒI DUAL-CHANNEL VÀ ĐẶC
TRƯNG PHIÊN POINT-IN-TIME**

Ngành:	Tài năng Khoa học máy tính
Thành viên thực hiện:	Đoàn Anh Hùng Nguyễn Quang Hiệp
Giảng viên hướng dẫn:	TS. Nguyễn Ngọc Giang

HÀ NỘI, 05/2026

Tóm tắt nghiên cứu

Báo cáo này tập trung nghiên cứu, thiết kế và phát triển hệ thống gợi ý sản phẩm thương mại điện tử hai giai đoạn (Two-Stage Recommender System) hướng tới độ trễ thấp ($< 50\text{ms}$) và khả năng cá nhân hóa theo hành vi người dùng. Hệ thống kết hợp giai đoạn tạo ứng viên (Recall/Candidate Generation) theo cơ chế triệu hồi song song hai kênh (Dual-Channel Recall) trên nền tảng Matrix Factorization của PyTorch cùng tìm kiếm vector lân cận gần nhất bằng thư viện FAISS, sau đó sử dụng giai đoạn xếp hạng (Ranking) bằng thuật toán học máy dạng cây LightGBM. Để giảm nguy cơ rò rỉ dữ liệu tương lai, đề tài áp dụng nguyên lý trích xuất đặc trưng theo thời điểm (Point-in-time Feature Engineering). Ngoài ra, vấn đề lệch phân phối giữa dữ liệu huấn luyện Ranking và dữ liệu khi phục vụ suy luận (train-serving mismatch) được giảm nhẹ thông qua kịch bản huấn luyện trên ứng viên thực tế (Train-on-Recall). Theo kết quả chạy lại trong môi trường cục bộ, mô hình cải tiến giúp AUC trên tập kiểm định đạt 0.9673 (+51.7%) và NDCG@10 đạt 0.0653 (+31.9%) trong bối cảnh dữ liệu tương tác thưa thớt.

Từ khóa: Hệ thống gợi ý hai giai đoạn, Tạo ứng viên, Xếp hạng, Matrix Factorization, FAISS, LightGBM, Point-in-time Feature Engineering, Train-on-Recall.

Mục lục

Tóm tắt nghiên cứu	i
1 Mở đầu	1
1.1 Lý do chọn đề tài	1
1.2 Mục tiêu của dự án	2
1.3 Phạm vi thực hiện	2
1.4 Công nghệ sử dụng	2
2 Cơ sở lý thuyết	4
2.1 Bài toán gợi ý sản phẩm trong thương mại điện tử	4
2.2 Phản hồi ẩn	4
2.3 Kiến trúc hệ gợi ý hai giai đoạn	5
2.4 Các kỹ thuật chính	6
2.5 Các chỉ số đánh giá	6
3 Phân tích dữ liệu và tiền xử lý	8
3.1 Mô tả dữ liệu đầu vào	8
3.2 Phân tích đặc điểm dữ liệu	9
3.3 Tiền xử lý dữ liệu	10
3.4 Gán nhãn và trọng số mẫu	10
3.5 Trích xuất đặc trưng và đặc trưng theo thời điểm	11
4 Thiết kế hệ thống	13
4.1 Tổng quan kiến trúc	13
4.2 Luồng xử lý đầu cuối	14
4.3 Cấu trúc mã nguồn	15
4.4 Các artifact đầu ra	16
5 Xây dựng mô hình gợi ý	17
5.1 Mô hình Recall	17
5.2 Matrix Factorization bằng PyTorch	18

5.3	Huấn luyện Recall và tìm kiếm vector bằng FAISS	18
5.4	Dual-channel Recall	19
5.5	Mô hình Ranking	20
5.6	Huấn luyện Ranking và Train-on-Recall	20
6	Triển khai API và phục vụ gợi ý	22
6.1	FastAPI server	22
6.2	Endpoint chính	22
6.3	Quy trình xử lý yêu cầu	23
6.4	Xử lý cold-start	24
6.5	Xử lý thiếu đặc trưng khi phục vụ	24
6.6	Đo độ trễ	25
7	Kết quả thực nghiệm và đánh giá	26
7.1	Kết quả pipeline dữ liệu	26
7.2	Kết quả Recall	27
7.3	Kết quả Ranking	28
7.4	Kết quả phục vụ suy luận	29
7.5	Nhận xét và hạn chế	30
8	Kết luận và hướng phát triển	32
8.1	Kết luận	32
8.2	Hướng phát triển	33
A	Phụ lục	37
A.1	Lệnh chạy chương trình	37
A.2	Ví dụ yêu cầu API	37
A.3	Ví dụ phản hồi JSON	37
A.4	Bảng siêu tham số	39

Danh sách hình vẽ

3.1	Phân phối loại hành vi theo phân tích EDA của dự án	9
3.2	Pipeline tiền xử lý dữ liệu	10
3.3	Minh họa nguyên tắc point-in-time khi tạo đặc trưng huấn luyện	12
4.1	Kiến trúc tổng thể của hệ thống gợi ý hai giai đoạn	14
4.2	Workflow vận hành thời gian thực	15
5.1	Sơ đồ Matrix Factorization	18
5.2	Cơ chế Dual-Channel Recall	20
5.3	Quy trình tạo dữ liệu Train-on-Recall	21
6.1	Luồng xử lý một yêu cầu gợi ý trong API FastAPI	24
7.1	Đường giảm loss của mô hình Recall qua 5 epoch	27
7.2	So sánh AUC, NDCG@10 và MRR giữa Baseline và Train-on-Recall	29
7.3	Biểu đồ phân rã độ trễ theo bảng tổng hợp pipeline phục vụ	30

Danh sách bảng

7.1	Kết quả huấn luyện và đánh giá mô hình Recall	27
7.2	Bảng so sánh hiệu năng xếp hạng giữa Baseline và Train-on-Recall	28
7.3	Bảng tổng hợp độ trễ tham chiếu của pipeline phục vụ	30
A.1	Một số tham số cấu hình chính của hệ thống	39

Chương 1

Mở đầu

Chương mở đầu trình bày bối cảnh hình thành đề tài, mục tiêu thực hiện, phạm vi triển khai và các công nghệ chính được sử dụng trong dự án. Nội dung của chương này giúp xác định rõ bài toán mà hệ thống cần giải quyết trước khi đi vào phân lý thuyết, dữ liệu, mô hình và triển khai kỹ thuật ở các chương sau.

1.1 Lý do chọn đề tài

Trong bối cảnh thương mại điện tử, số lượng sản phẩm mà người dùng có thể tiếp cận thường rất lớn, trong khi nhu cầu và hành vi mua sắm của từng người lại khác nhau. Nếu chỉ dựa vào tìm kiếm thủ công hoặc danh sách sản phẩm phổ biến, hệ thống khó phản ánh được sở thích riêng của từng người dùng. Vì vậy, cá nhân hóa trở thành một yêu cầu quan trọng để cải thiện trải nghiệm duyệt và lựa chọn sản phẩm [1].

Hệ thống gợi ý sản phẩm giải quyết vấn đề này bằng cách khai thác lịch sử tương tác như xem sản phẩm, thêm vào giỏ hàng hoặc mua hàng để ước lượng mức độ phù hợp giữa người dùng và sản phẩm. Với cách tiếp cận đó, hệ thống có thể rút ngắn quá trình tìm kiếm của người dùng, đồng thời giúp nền tảng khai thác dữ liệu hành vi một cách có cấu trúc hơn.

Tuy nhiên, dữ liệu tương tác trong thương mại điện tử thường thưa và mất cân bằng: phần lớn sự kiện là lượt xem, còn các hành vi có giá trị cao như thêm vào giỏ hàng hoặc mua hàng xuất hiện ít hơn. Bên cạnh đó, hệ thống phục vụ trực tuyến phải trả kết quả trong thời gian ngắn. Vì vậy, đề tài này lựa chọn kiến trúc gợi ý hai giai đoạn (Two-Stage Recommender System), gồm bước tạo ứng viên (Recall/Candidate Generation) và bước xếp hạng (Ranking), tương tự cách tổ chức thường gặp trong các hệ gợi ý quy mô lớn [1].

1.2 Mục tiêu của dự án

Mục tiêu chính của dự án là xây dựng một hệ thống gợi ý sản phẩm cho thương mại điện tử có khả năng xử lý dữ liệu hành vi người dùng, huấn luyện mô hình học máy và phục vụ kết quả gợi ý thông qua API. Hệ thống được thiết kế theo kiến trúc hai giai đoạn nhằm cân bằng giữa tốc độ xử lý và chất lượng gợi ý.

Ở giai đoạn dữ liệu, dự án xây dựng quy trình xử lý dữ liệu (Data Pipeline) từ dữ liệu thô, chia phiên truy cập của người dùng dựa trên khoảng cách thời gian giữa các hành vi, đồng thời gán nhãn và trọng số cho các loại tương tác như `view`, `cart`, `purchase`. Từ dữ liệu đã xử lý, hệ thống tạo ra các đặc trưng mô tả người dùng, sản phẩm và phiên truy cập để phục vụ quá trình huấn luyện mô hình.

Ở giai đoạn mô hình, dự án xây dựng mô hình Recall bằng Matrix Factorization nhằm lọc nhanh các sản phẩm ứng viên từ tập sản phẩm lớn [2]. Các vector biểu diễn sản phẩm sau đó được đưa vào FAISS để hỗ trợ tìm kiếm tương đồng tốc độ cao [3]. Trên tập ứng viên này, mô hình Ranking bằng LightGBM được sử dụng để chấm điểm và sắp xếp lại danh sách sản phẩm [4]. Cuối cùng, hệ thống được triển khai thông qua FastAPI để trả về danh sách Top-N sản phẩm gợi ý cho từng người dùng [5].

1.3 Phạm vi thực hiện

Trong phạm vi bài tập lớn, dự án tập trung vào việc xây dựng một hệ thống gợi ý hoàn chỉnh ở mức thực nghiệm, bao gồm các bước từ xử lý dữ liệu, huấn luyện mô hình đến triển khai API phục vụ gợi ý. Dữ liệu được sử dụng là dữ liệu hành vi thương mại điện tử dạng offline, bao gồm các tương tác của người dùng với sản phẩm theo thời gian.

Hệ thống tập trung vào bài toán gợi ý sản phẩm dựa trên hành vi người dùng và chưa đi sâu vào các thành phần hạ tầng phức tạp như xử lý dữ liệu streaming, cập nhật mô hình liên tục theo thời gian thực, kho đặc trưng (feature store) chuyên dụng hoặc A/B testing trên người dùng thật. Các kết quả đánh giá trong báo cáo chủ yếu dựa trên đánh giá offline và kiểm thử API trong môi trường cục bộ.

1.4 Công nghệ sử dụng

Dự án được xây dựng bằng ngôn ngữ Python và sử dụng nhiều thư viện phục vụ xử lý dữ liệu, học máy và triển khai API. pandas và NumPy được dùng để xử lý dữ liệu dạng bảng, tính toán đặc trưng và thao tác với mảng số. scikit-learn được sử dụng cho các bước hỗ trợ như mã hóa định danh người dùng và sản phẩm.

Ở phần mô hình, PyTorch được dùng để xây dựng và huấn luyện mô hình Matrix Factorization cho giai đoạn Recall [6], trong khi FAISS hỗ trợ xây dựng chỉ mục vector và tìm kiếm sản phẩm tương đồng tốc độ cao [3]. LightGBM được sử dụng ở giai đoạn Ranking để chấm điểm và sắp xếp lại các ứng viên [4]. Đối với phần triển khai, FastAPI và Uvicorn được dùng để xây dựng API phục vụ gợi ý sản phẩm [5]. Ngoài ra, dự án sử dụng Parquet để lưu trữ dữ liệu dạng cột [7] và joblib để lưu encoder, artifact của mô hình; công cụ uv được dùng để quản lý môi trường và chạy các lệnh trong dự án.

Chương 2

Cơ sở lý thuyết

Chương này trình bày các khái niệm nền tảng cần thiết để hiểu cách hệ thống gợi ý trong dự án được thiết kế. Các nội dung được triển khai theo thứ tự từ bài toán gợi ý sản phẩm, phản hồi ẩn, kiến trúc hai giai đoạn, các kỹ thuật chính cho đến các chỉ số đánh giá mô hình.

2.1 Bài toán gợi ý sản phẩm trong thương mại điện tử

Bài toán gợi ý sản phẩm trong thương mại điện tử có thể được xem là bài toán chọn ra một danh sách sản phẩm phù hợp cho từng người dùng từ một tập sản phẩm lớn. Khác với tìm kiếm theo từ khóa, hệ thống gợi ý phải tận dụng các tín hiệu hành vi để suy ra sở thích tiềm ẩn của người dùng và cá nhân hóa danh sách hiển thị [1].

Mục tiêu của hệ thống gợi ý là khai thác dữ liệu hành vi trong quá khứ để dự đoán những sản phẩm mà người dùng có khả năng quan tâm trong tương lai [1]. Với một người dùng cụ thể, hệ thống cần đưa ra danh sách Top-N sản phẩm phù hợp nhất. Danh sách này có thể được sử dụng ở nhiều vị trí trong ứng dụng thương mại điện tử như trang chủ cá nhân hóa, trang chi tiết sản phẩm, giỏ hàng hoặc các chiến dịch tiếp thị.

Trong phạm vi dự án này, dữ liệu đầu vào là lịch sử tương tác giữa người dùng và sản phẩm. Mỗi tương tác thể hiện một hành vi như xem sản phẩm, thêm vào giỏ hàng hoặc mua hàng. Từ dữ liệu đó, hệ thống xây dựng mô hình học máy nhằm học được mối quan hệ giữa người dùng và sản phẩm, sau đó sử dụng mô hình để sinh ra danh sách sản phẩm gợi ý.

2.2 Phản hồi ẩn

Trong các hệ thống gợi ý, dữ liệu phản hồi của người dùng thường được phân biệt thành phản hồi tường minh (explicit feedback) và phản hồi ẩn (implicit feedback). Phản hồi tường minh

là các tín hiệu trực tiếp như đánh giá sao, lượt thích hoặc bình luận. Ngược lại, phản hồi ẩn được suy ra từ hành vi sử dụng hệ thống, chẳng hạn lượt xem, lượt thêm vào giỏ hàng hoặc lượt mua [8]. Trong thương mại điện tử, dạng phản hồi ẩn thường phổ biến hơn vì không phải người dùng nào cũng để lại đánh giá trực tiếp.

Trong dự án này, phản hồi ẩn được biểu diễn thông qua các sự kiện hành vi gồm `view`, `cart` và `purchase`. Hành vi `view` cho thấy người dùng đã quan tâm ở mức độ thấp vì họ chỉ xem sản phẩm. Hành vi `cart` thể hiện mức độ quan tâm cao hơn vì người dùng đã đưa sản phẩm vào giỏ hàng. Hành vi `purchase` là tín hiệu mạnh nhất, cho thấy người dùng thực sự có nhu cầu và đã hoàn tất giao dịch.

Một đặc điểm quan trọng của phản hồi ẩn là việc không có tương tác không đồng nghĩa với việc người dùng không thích sản phẩm. Người dùng có thể chưa nhìn thấy sản phẩm, hoặc sản phẩm chưa xuất hiện trong ngữ cảnh mua sắm hiện tại. Do đó, khi xây dựng dữ liệu huấn luyện từ phản hồi ẩn, bước gán nhãn và lấy mẫu âm cần được thiết kế cẩn thận để hạn chế làm sai lệch quá trình học mô hình [8].

2.3 Kiến trúc hệ gợi ý hai giai đoạn

Trong các hệ thống thương mại điện tử thực tế, số lượng sản phẩm có thể rất lớn. Nếu hệ thống trực tiếp chấm điểm toàn bộ sản phẩm cho từng người dùng, chi phí tính toán sẽ cao và khó đáp ứng yêu cầu thời gian thực. Vì vậy, nhiều hệ thống gợi ý hiện đại sử dụng kiến trúc hai giai đoạn (Two-Stage Recommender System) [1].

Giai đoạn đầu tiên là tạo ứng viên (Recall/Candidate Generation). Nhiệm vụ của giai đoạn này là lọc nhanh một tập ứng viên nhỏ hơn từ toàn bộ tập sản phẩm. Recall không cần sắp xếp hoàn hảo tất cả sản phẩm, nhưng cần bảo đảm rằng các sản phẩm có khả năng phù hợp cao được giữ lại trong tập ứng viên. Trong dự án này, Recall được xây dựng bằng mô hình Matrix Factorization sử dụng PyTorch [6]. Sau khi huấn luyện, vector embedding của sản phẩm được đưa vào FAISS để hỗ trợ tìm kiếm nhanh các sản phẩm gần nhất trong không gian vector.

Giai đoạn thứ hai là xếp hạng (Ranking). Sau khi Recall đã thu hẹp không gian tìm kiếm, Ranking sẽ chấm điểm chi tiết từng ứng viên dựa trên nhiều đặc trưng hơn, bao gồm đặc trưng người dùng, đặc trưng sản phẩm, đặc trưng phiên và thông tin nguồn Recall. Trong dự án này, mô hình Ranking sử dụng LightGBM để dự đoán mức độ phù hợp của từng ứng viên, sau đó sắp xếp theo điểm giảm dần và trả về danh sách Top-N sản phẩm [4].

2.4 Các kỹ thuật chính

Matrix Factorization là một kỹ thuật quan trọng trong dự án. Kỹ thuật này biểu diễn người dùng và sản phẩm dưới dạng các vector có số chiều thấp hơn so với ma trận tương tác ban đầu [2]. Thay vì thao tác trực tiếp trên toàn bộ ma trận người dùng-sản phẩm vốn rất thưa, mô hình học một vector embedding cho mỗi người dùng và mỗi sản phẩm. Mức độ phù hợp giữa hai đối tượng sau đó được tính thông qua phép nhân vô hướng giữa hai vector.

Dữ liệu thương mại điện tử thường mất cân bằng mạnh, trong đó phần lớn tương tác là lượt xem, còn hành vi mua hàng chiếm tỷ lệ nhỏ. Để giảm ảnh hưởng của mất cân bằng dữ liệu, dự án sử dụng Focal Loss trong quá trình huấn luyện mô hình Recall [9]. Focal Loss giúp mô hình tập trung nhiều hơn vào các mẫu khó và giảm ảnh hưởng của những mẫu dễ, từ đó phù hợp với bối cảnh có nhiều tín hiệu yếu và ít tín hiệu mạnh.

FAISS được sử dụng để tìm kiếm vector tốc độ cao [3]. Sau khi mô hình Recall học được item embedding, các vector này được lưu lại và xây dựng thành chỉ mục tìm kiếm. Khi cần gợi ý cho một người dùng, hệ thống chỉ cần lấy vector đại diện cho người dùng hoặc phiên hiện tại, sau đó truy vấn FAISS để lấy ra các sản phẩm gần nhất.

LightGBM được sử dụng ở giai đoạn Ranking [4]. Đây là mô hình Gradient Boosting Decision Trees có khả năng xử lý tốt dữ liệu dạng bảng và các đặc trưng hỗn hợp. Trong dự án này, LightGBM nhận đầu vào là các đặc trưng đã được tạo từ pipeline dữ liệu và kho đặc trưng (feature store), sau đó dự đoán điểm phù hợp cho từng sản phẩm ứng viên.

2.5 Các chỉ số đánh giá

Để đánh giá hệ thống gợi ý, dự án sử dụng nhiều nhóm chỉ số khác nhau vì mỗi giai đoạn có mục tiêu riêng. Đối với giai đoạn Recall, chỉ số quan trọng là Recall@K hoặc Hit Rate@K. Chỉ số này phản ánh khả năng của mô hình trong việc giữ lại các sản phẩm liên quan trong Top-K ứng viên [1].

Đối với giai đoạn Ranking, AUC được sử dụng để đánh giá khả năng phân biệt giữa mẫu tích cực và mẫu không tích cực [10]. Tuy nhiên, vì bài toán gợi ý quan tâm nhiều đến thứ tự hiển thị, các chỉ số xếp hạng như NDCG@K và MRR cũng rất quan trọng. NDCG@K đánh giá chất lượng thứ tự của danh sách Top-K, trong đó các sản phẩm phù hợp xuất hiện ở vị trí cao sẽ đóng góp nhiều hơn vào điểm số [11]. MRR đo vị trí của sản phẩm phù hợp đầu tiên trong danh sách, qua đó phản ánh mức độ nhanh chóng mà người dùng có thể nhìn thấy một sản phẩm liên quan [12].

Bên cạnh chất lượng gợi ý, hệ thống còn cần được đánh giá về hiệu năng phục vụ. Chỉ số độ trễ (latency) cho biết thời gian hệ thống cần để xử lý một request và trả về kết quả. Trong

hệ thống gợi ý trực tuyến, latency là yếu tố quan trọng vì thời gian phản hồi chậm có thể ảnh hưởng trực tiếp đến trải nghiệm người dùng.

Chương 3

Phân tích dữ liệu và tiền xử lý

Chương này tập trung phân tích dữ liệu đầu vào và mô tả cách dự án chuyển đổi dữ liệu hành vi thô thành dữ liệu có thể sử dụng cho huấn luyện và phục vụ suy luận. Trình tự trình bày đi từ mô tả dữ liệu, nhận diện đặc điểm dữ liệu, tiền xử lý, gán nhãn đến trích xuất đặc trưng theo thời điểm.

3.1 Mô tả dữ liệu đầu vào

Dữ liệu đầu vào của dự án là dữ liệu hành vi người dùng trong môi trường thương mại điện tử. Mỗi dòng dữ liệu biểu diễn một sự kiện tương tác giữa một người dùng và một sản phẩm tại một thời điểm cụ thể. Đây là dạng dữ liệu phổ biến trong các hệ thống gợi ý sản phẩm, vì nó phản ánh trực tiếp quá trình người dùng duyệt, cân nhắc và mua hàng trên nền tảng.

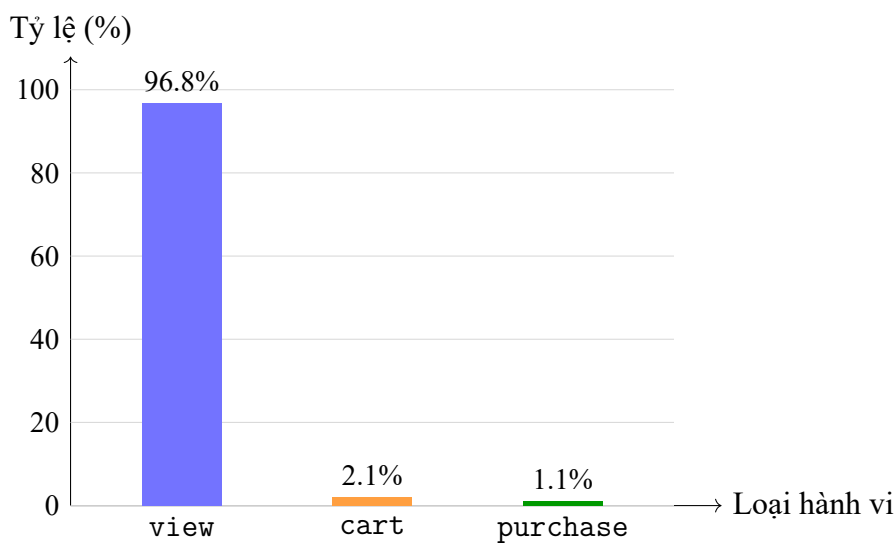
Các trường dữ liệu quan trọng bao gồm `event_time`, `event_type`, `product_id`, `user_id`, `category_code`, `brand` và `price`. Trong đó, `event_time` cho biết thời điểm xảy ra hành vi, giúp hệ thống sắp xếp các tương tác theo trình tự thời gian và chia phiên người dùng. Trường `event_type` mô tả loại hành vi, chẳng hạn như xem sản phẩm, thêm vào giỏ hàng hoặc mua sản phẩm. Hai trường `user_id` và `product_id` lần lượt định danh người dùng và sản phẩm, đóng vai trò trung tâm trong việc xây dựng ma trận tương tác người dùng-sản phẩm (user-item).

Ngoài các trường định danh và hành vi, dữ liệu còn có các thuộc tính mô tả sản phẩm như `category_code`, `brand` và `price`. Các thuộc tính này cung cấp thêm ngữ cảnh cho mô hình Ranking, giúp mô hình không chỉ dựa vào lịch sử tương tác mà còn khai thác thông tin về danh mục, thương hiệu và mức giá của sản phẩm.

3.2 Phân tích đặc điểm dữ liệu

Dữ liệu thương mại điện tử có một số đặc điểm gây khó khăn cho bài toán gợi ý. Đặc điểm đầu tiên là độ thưa cao của ma trận tương tác giữa người dùng và sản phẩm. Trong thực tế, mỗi người dùng chỉ tương tác với một số lượng rất nhỏ sản phẩm so với toàn bộ danh mục sản phẩm của hệ thống. Điều này khiến ma trận người dùng-sản phẩm có phần lớn giá trị trống, làm cho việc học sở thích người dùng trở nên khó khăn nếu sử dụng các phương pháp đơn giản.

Đặc điểm thứ hai là sự mất cân bằng trong phân phối hành vi. Hành vi `view` thường chiếm tỷ lệ lớn vì người dùng có xu hướng xem nhiều sản phẩm trước khi ra quyết định. Ngược lại, các hành vi thể hiện mức độ quan tâm cao hơn như `cart` và `purchase` thường xuất hiện ít hơn nhiều. Mức chênh lệch này được minh họa trong Hình 3.1. Nếu không xử lý vấn đề này, mô hình có thể bị thiên lệch về lớp đa số và không học tốt các tín hiệu quan trọng liên quan đến ý định mua hàng.



Hình 3.1: Phân phối loại hành vi theo phân tích EDA của dự án

Từ Hình 3.1, có thể thấy `view` chiếm gần như toàn bộ dữ liệu tương tác, trong khi `cart` và `purchase` chỉ chiếm tỷ lệ nhỏ. Điều này giải thích vì sao báo cáo cần trình bày riêng cơ chế gán nhãn và trọng số mẫu, bởi nếu xem mọi hành vi như nhau thì tín hiệu mua hàng sẽ bị lấn át bởi lượng lớn lượt xem sản phẩm.

Đặc điểm thứ ba là dữ liệu có thể chứa giá trị thiếu ở các trường phân loại như `brand` hoặc `category_code`. Đây là vấn đề thường gặp trong dữ liệu sản phẩm, đặc biệt khi danh mục hàng hóa lớn và thông tin sản phẩm không được chuẩn hóa hoàn toàn. Ngoài ra, trường

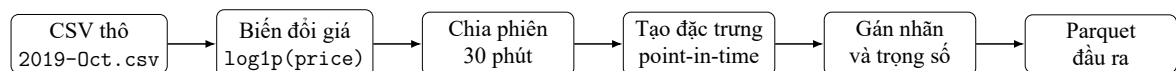
price thường có phân phối lệch, vì một số sản phẩm có giá rất cao so với phần lớn sản phẩm còn lại.

3.3 Tiền xử lý dữ liệu

Quá trình tiền xử lý dữ liệu bắt đầu từ việc đọc dữ liệu thô từ file CSV. Trong cấu hình của dự án, đường dẫn dữ liệu thô được khai báo tại `RAW_DATA_PATH`, tương ứng với file `data/raw/2019-Oct.csv`. Để phù hợp với điều kiện thực nghiệm và giới hạn tài nguyên, pipeline có thể đọc một số lượng dòng nhất định thông qua tham số `PIPELINE_NROWS`.

Sau khi đọc dữ liệu, trường `event_time` được chuyển sang kiểu thời gian để có thể sắp xếp và tính toán khoảng cách giữa các sự kiện. Đây là bước cần thiết vì nhiều thao tác sau đó, đặc biệt là chia phiên và tính đặc trưng `point-in-time`, đều phụ thuộc vào trình tự thời gian của các hành vi. Dữ liệu sau đó được sắp xếp theo `user_id` và `event_time` để bảo đảm các tương tác của từng người dùng được xử lý đúng thứ tự.

Đối với trường `price`, dự án sử dụng phép biến đổi `log1p`, tức là tính $\log(1 + price)$. Cách biến đổi này giúp giảm ảnh hưởng của các giá trị ngoại lai và làm cho phân phối giá bớt lệch. Một bước quan trọng khác là chia phiên người dùng. Trong dự án, các hành vi liên tiếp của cùng một người dùng được xem là thuộc cùng một phiên nếu khoảng cách thời gian giữa chúng không vượt quá 30 phút. Nếu khoảng cách lớn hơn ngưỡng này, hệ thống coi đó là một phiên mới. Toàn bộ chuỗi xử lý chính của pipeline được tóm tắt trong Hình 3.2.



Hình 3.2: Pipeline tiền xử lý dữ liệu của dự án

Hình 3.2 cho thấy pipeline không chỉ làm sạch dữ liệu mà còn chuẩn bị trực tiếp đầu vào cho hai giai đoạn mô hình. Kết quả cuối cùng được lưu dưới dạng Parquet để các bước huấn luyện Recall, huấn luyện Ranking và phục vụ API có thể dùng chung một nguồn dữ liệu đã chuẩn hóa.

3.4 Gán nhãn và trọng số mẫu

Dữ liệu hành vi ban đầu chưa thể đưa trực tiếp vào mô hình học máy vì mỗi dòng chỉ mô tả một loại sự kiện. Do đó, dự án cần chuyển đổi các hành vi này thành nhãn huấn luyện. Cách

tiếp cận được sử dụng là xem `view` như một tín hiệu yếu và gán nhãn 0, trong khi `cart` và `purchase` được xem là tín hiệu tích cực và gán nhãn 1.

Tuy nhiên, không phải mọi hành vi đều có mức độ quan trọng như nhau. Một lượt xem sản phẩm chỉ cho biết người dùng đã tiếp xúc với sản phẩm, nhưng chưa chắc họ thực sự có ý định mua. Trong khi đó, hành vi thêm vào giỏ hàng và mua hàng thể hiện mức độ quan tâm cao hơn. Vì vậy, dự án sử dụng thêm `sample_weight` để biểu diễn cường độ của từng hành vi. Cụ thể, `view` có trọng số 0.1, `cart` có trọng số 0.5 và `purchase` có trọng số 1.0.

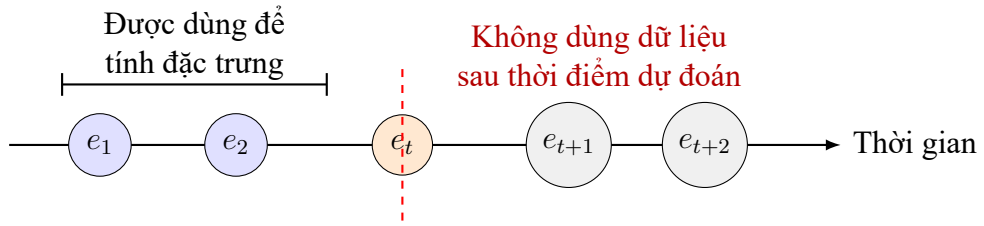
Trong quá trình gán nhãn, dữ liệu được gom theo `custom_session_id`, `product_id` và `user_id`. Nếu trong cùng một phiên, cùng một người dùng tương tác nhiều lần với cùng một sản phẩm, hệ thống giữ lại hành vi mạnh nhất. Cách xử lý này giúp giảm trùng lặp và làm cho nhãn huấn luyện phản ánh tốt hơn mức độ quan tâm thực sự của người dùng.

3.5 Trích xuất đặc trưng và đặc trưng theo thời điểm

Trích xuất đặc trưng (Feature Engineering) là bước tạo ra các đặc trưng đầu vào cho mô hình Ranking và hỗ trợ quá trình phục vụ gợi ý. Trong dự án, các đặc trưng được chia thành nhiều nhóm, bao gồm đặc trưng người dùng, đặc trưng sản phẩm, đặc trưng phiên và đặc trưng nguồn Recall.

Nhóm đặc trưng người dùng mô tả mức độ hoạt động tổng quát của mỗi người dùng, chẳng hạn tổng số tương tác trước đó và số phiên đã tham gia. Nhóm đặc trưng sản phẩm mô tả độ phổ biến, số lượng người dùng từng quan tâm, giá trung bình và độ phổ biến theo phiên. Nhóm đặc trưng phiên giúp mô hình nắm bắt ngữ cảnh ngắn hạn của người dùng tại thời điểm đang xét.

Một vấn đề quan trọng trong xây dựng hệ thống gợi ý là rò rỉ dữ liệu tương lai. Nếu các đặc trưng được tính bằng cách tổng hợp toàn bộ dữ liệu, mô hình có thể vô tình sử dụng thông tin xảy ra sau thời điểm cần dự đoán. Để hạn chế vấn đề này, dự án xây dựng các đặc trưng theo nguyên tắc theo thời điểm (point-in-time). Điều đó có nghĩa là tại mỗi dòng dữ liệu, các đặc trưng chỉ được tính từ những sự kiện đã xảy ra trước thời điểm của dòng đó, như minh họa trong Hình 3.3.



Hình 3.3: Minh họa nguyên tắc point-in-time khi tạo đặc trưng huấn luyện

Hình 3.3 nhấn mạnh ranh giới thời gian tại sự kiện cần dự đoán. Các sự kiện phía trước ranh giới được phép dùng để tạo đặc trưng, còn các sự kiện phía sau phải bị loại khỏi phép tổng hợp. Cách trình bày này giúp làm rõ vì sao đặc trưng theo thời điểm quan trọng đối với tính khách quan của đánh giá mô hình.

Sau khi hoàn thành bước xử lý dữ liệu, pipeline lưu ba nhóm dữ liệu chính ra đĩa. Tập `labeled_sessions.parquet` chứa dữ liệu huấn luyện đã được gán nhãn và bổ sung đặc trưng. Tập `user_features.parquet` và `item_features.parquet` đóng vai trò kho đặc trưng dạng snapshot (feature store snapshot), được sử dụng ở giai đoạn phục vụ API để trả cứu nhanh thông tin người dùng và sản phẩm.

Chương 4

Thiết kế hệ thống

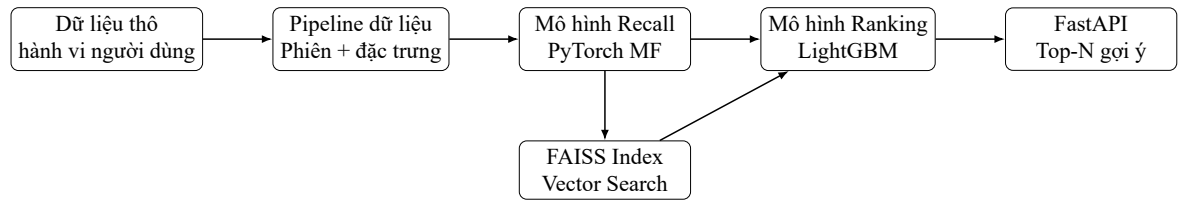
Chương này mô tả thiết kế tổng thể của hệ thống sau khi dữ liệu và yêu cầu bài toán đã được xác định. Nội dung tập trung vào kiến trúc hai giai đoạn, luồng xử lý đầu cuối, cấu trúc mã nguồn và các artifact được tạo ra để kết nối giữa huấn luyện mô hình và phục vụ API.

4.1 Tổng quan kiến trúc

Hệ thống được thiết kế theo kiến trúc gợi ý hai giai đoạn, bao gồm giai đoạn xử lý dữ liệu, giai đoạn huấn luyện mô hình và giai đoạn phục vụ gợi ý thông qua API. Cách tổ chức này giúp tách biệt rõ ràng giữa phần chuẩn bị dữ liệu, phần học mô hình và phần suy luận trực tuyến, từ đó làm cho hệ thống dễ mở rộng, dễ kiểm thử và dễ thay thế từng thành phần khi cần cải tiến.

Ở mức tổng quan, dữ liệu thô được đưa vào quy trình xử lý dữ liệu (Data Pipeline) để thực hiện các bước tiền xử lý, chia phiên, tạo đặc trưng và gán nhãn. Kết quả của pipeline được lưu lại dưới dạng các tập dữ liệu Parquet, bao gồm dữ liệu huấn luyện và các bảng đặc trưng dùng cho giai đoạn phục vụ suy luận. Các bảng đặc trưng này đóng vai trò như một kho đặc trưng (feature store) đơn giản, cho phép hệ thống tra cứu nhanh thông tin người dùng và sản phẩm trong quá trình phục vụ gợi ý.

Sau khi dữ liệu đã được xử lý, hệ thống huấn luyện mô hình Recall bằng Matrix Factorization. Mô hình này học biểu diễn vector cho người dùng và sản phẩm. Vector sản phẩm sau huấn luyện được lưu thành file embedding và đưa vào FAISS để xây dựng chỉ mục tìm kiếm vector. Khi cần sinh gợi ý, FAISS giúp truy hồi nhanh một tập ứng viên sản phẩm thay vì phải xét toàn bộ danh mục. Hình 4.1 tóm tắt các thành phần chính và quan hệ giữa chúng ở mức kiến trúc.



Hình 4.1: Kiến trúc tổng thể của hệ thống gợi ý hai giai đoạn

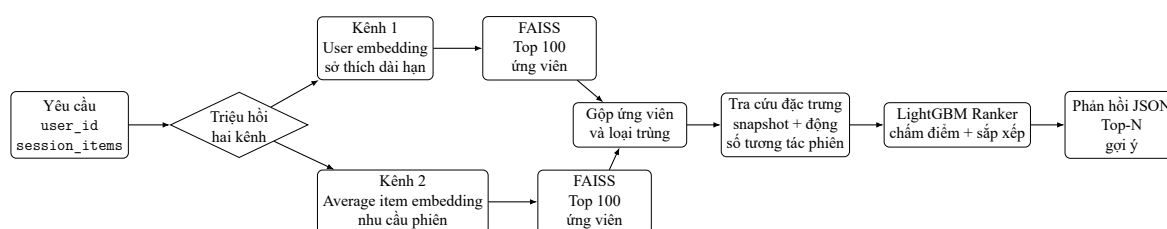
Từ Hình 4.1, có thể thấy kiến trúc được tách thành ba vùng trách nhiệm rõ ràng: chuẩn bị dữ liệu, huấn luyện mô hình và phục vụ gợi ý. Cách tách này phù hợp với cấu trúc mã nguồn của dự án, trong đó mỗi nhóm thư mục đảm nhiệm một phần riêng nhưng vẫn dùng chung các artifact đã được sinh ra từ pipeline.

4.2 Luồng xử lý đầu cuối

Luồng xử lý đầu cuối (end-to-end) của hệ thống bắt đầu từ dữ liệu hành vi thô và kết thúc ở danh sách sản phẩm gợi ý được trả về cho người dùng. Bước phân đoạn phiên (Sessionization) có nhiệm vụ chia các hành vi liên tiếp của người dùng thành các phiên mua sắm. Đây là bước quan trọng vì hành vi trong cùng một phiên thường phản ánh nhu cầu ngắn hạn của người dùng. Sau đó, bước trích xuất đặc trưng (Feature Engineering) tạo ra các đặc trưng mô tả người dùng, sản phẩm và phiên. Các đặc trưng này được sử dụng trong cả quá trình huấn luyện và phục vụ mô hình.

Gán nhãn giả (*Pseudo-Labeling*) chuyển đổi dữ liệu hành vi thành nhãn huấn luyện. Thay vì chỉ lưu lại loại sự kiện ban đầu, hệ thống ánh xạ các hành vi thành nhãn và trọng số mẫu để mô hình có thể học được mức độ quan tâm của người dùng. Dữ liệu đã gán nhãn được dùng để huấn luyện mô hình Recall và Ranking.

Sau khi mô hình Recall được huấn luyện, hệ thống xuất item embedding và xây dựng FAISS index. Đây là thành phần hỗ trợ truy hồi ứng viên tốc độ cao. Ở giai đoạn Ranking, LightGBM được huấn luyện trên các đặc trưng đã tạo để học cách sắp xếp các sản phẩm ứng viên. Khi triển khai API, pipeline gợi ý sẽ kết hợp FAISS, kho đặc trưng và LightGBM để trả về danh sách sản phẩm cuối cùng. Quy trình vận hành trực tuyến này được mô tả cụ thể hơn trong Hình 4.2.



Hình 4.2: Workflow vận hành thời gian thực của hệ thống, được trực quan hóa từ sơ đồ Architecture Workflow trong README

Hình 4.2 làm rõ điểm khác biệt giữa kiến trúc tổng thể và luồng vận hành khi có request thực tế. Ở thời điểm phục vụ, hệ thống không chạy lại toàn bộ pipeline dữ liệu mà sử dụng các artifact đã có sẵn để truy hồi ứng viên qua hai kênh, bổ sung đặc trưng, chấm điểm bằng LightGBM và trả về danh sách Top-N dưới dạng JSON.

4.3 Cấu trúc mã nguồn

Mã nguồn của dự án được tổ chức theo hướng mô-đun, trong đó mỗi thư mục chịu trách nhiệm cho một phần riêng của hệ thống. Cách tổ chức này giúp quá trình phát triển và kiểm thử rõ ràng hơn, đồng thời phản ánh đúng các giai đoạn chính trong pipeline gợi ý.

File `src/config.py` đóng vai trò là nơi cấu hình tập trung của dự án. File này khai báo các đường dẫn dữ liệu, đường dẫn lưu mô hình, siêu tham số (hyperparameters) cho Recall và Ranking, cũng như danh sách đặc trưng số và đặc trưng phân loại được sử dụng trong mô hình. Việc tập trung cấu hình tại một nơi giúp hạn chế sai lệch giữa các giai đoạn xử lý dữ liệu, huấn luyện và phục vụ suy luận.

Thư mục `src/data_pipeline` chứa các thành phần xử lý dữ liệu. Trong đó, module `sessionizer` thực hiện chia phiên người dùng dựa trên khoảng cách thời gian giữa các sự kiện, module `featurizer` tạo các đặc trưng theo thời điểm (point-in-time features) và snapshot đặc trưng, còn module `pseudo-label` chuyển đổi hành vi người dùng thành nhãn và trọng số huấn luyện. File điều phối pipeline chịu trách nhiệm chạy toàn bộ các bước này để tạo ra dữ liệu đầu ra phục vụ các giai đoạn tiếp theo.

Thư mục `src/recall_model` chứa phần xây dựng mô hình Recall. Mô hình Matrix Factorization được định nghĩa trong `model.py`, quá trình huấn luyện và hàm mất mát được triển khai trong `trainer.py`, còn `run_recall.py` thực hiện toàn bộ quy trình đọc dữ liệu, lấy mẫu âm, mã hóa định danh, huấn luyện mô hình và lưu các artifact.

Thư mục `src/ranking_model` phụ trách mô hình Ranking. File `lgbm_train.py` định nghĩa lớp huấn luyện LightGBM, `metrics.py` triển khai các chỉ số đánh giá như NDCG

và MRR, còn `run_ranking.py` huấn luyện mô hình Ranking baseline từ dữ liệu đã xử lý. Ngoài ra, `train_on_recall.py` triển khai hướng cải tiến bằng cách huấn luyện Ranking trên chính các ứng viên được sinh ra từ Recall.

Thư mục `src/serving` chứa các thành phần phục vụ gợi ý. `faiss_index.py` quản lý việc tải item embedding và xây dựng FAISS index. `ranker.py` tải mô hình LightGBM và thực hiện dự đoán điểm cho các ứng viên. `pipeline.py` là thành phần điều phối chính, kết hợp Recall, tra cứu đặc trưng, Ranking và xử lý fallback để tạo danh sách gợi ý cuối cùng.

4.4 Các artifact đầu ra

Sau khi chạy đầy đủ pipeline, hệ thống tạo ra nhiều artifact phục vụ cho cả huấn luyện và suy luận. Tập `labeled_sessions.parquet` chứa dữ liệu tương tác đã được chia phiên, bổ sung đặc trưng và gán nhãn. Đây là tập dữ liệu chính dùng để huấn luyện mô hình Recall và Ranking. Hai tập `user_features.parquet` và `item_features.parquet` là các bảng snapshot đặc trưng cho người dùng và sản phẩm. Trong quá trình phục vụ suy luận, các bảng này được nạp vào bộ nhớ để tra cứu nhanh khi cần chấm điểm các ứng viên.

Đối với mô hình Recall, hệ thống lưu trọng số mô hình vào `recall_weights.pth`, lưu vector biểu diễn sản phẩm vào `item_embeddings.npy`, đồng thời lưu encoder ánh xạ giữa định danh gốc và chỉ số nội bộ vào `user_encoder.joblib` và `item_encoder.joblib`. Các file này cần được đồng bộ với nhau, vì FAISS index, embedding và mapping sản phẩm phải sử dụng cùng một thứ tự item.

Đối với mô hình Ranking, artifact quan trọng nhất là `ranker_model.txt`, chứa mô hình LightGBM đã huấn luyện. Khi API khởi động, `LightGBMRanker` tải file này và đọc danh sách đặc trưng mà mô hình yêu cầu. Nhờ đó, pipeline phục vụ suy luận có thể tự động căn chỉnh đặc trưng đầu vào, bổ sung các cột bị thiếu và đảm bảo thứ tự cột phù hợp với mô hình đã huấn luyện.

Chương 5

Xây dựng mô hình gợi ý

Chương này trình bày cách dự án xây dựng hai thành phần mô hình chính của hệ thống gợi ý: Recall và Ranking. Phần Recall chịu trách nhiệm tạo tập ứng viên nhanh bằng Matrix Factorization và FAISS, trong khi phần Ranking sử dụng LightGBM để chấm điểm và sắp xếp lại các ứng viên theo mức độ phù hợp.

5.1 Mô hình Recall

Trong kiến trúc hệ gợi ý hai giai đoạn, mô hình Recall đóng vai trò thu hẹp không gian tìm kiếm bằng cách lấy ra một tập sản phẩm ứng viên từ toàn bộ danh mục. Đây là bước đầu tiên trong quá trình gợi ý và có ảnh hưởng trực tiếp đến chi phí suy luận của giai đoạn Ranking. Nếu hệ thống có hàng chục nghìn hoặc hàng triệu sản phẩm, việc đưa toàn bộ sản phẩm vào mô hình Ranking sẽ tốn nhiều chi phí tính toán và không phù hợp với yêu cầu phản hồi nhanh [1].

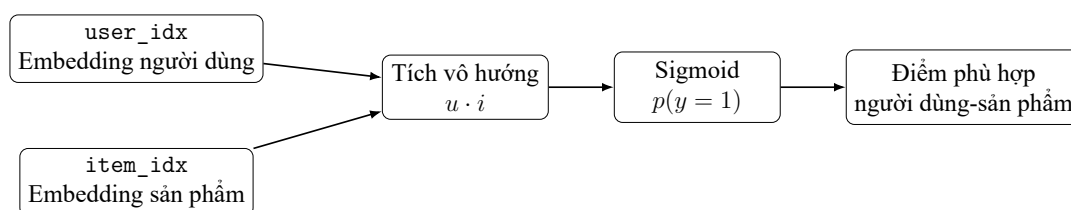
Ở giai đoạn này, mục tiêu chính không phải là tạo ra thứ tự cuối cùng hoàn hảo cho tất cả sản phẩm, mà là giữ lại đủ nhiều ứng viên có khả năng phù hợp. Nói cách khác, Recall ưu tiên độ bao phủ của các sản phẩm tiềm năng. Sau đó, mô hình Ranking sẽ phân tích chi tiết hơn và quyết định thứ tự cuối cùng của danh sách gợi ý.

Trong dự án này, Recall được xây dựng dựa trên Matrix Factorization [2] và được triển khai bằng PyTorch [6]. Mô hình học biểu diễn vector cho người dùng và sản phẩm từ dữ liệu tương tác lịch sử. Khi cần gợi ý, hệ thống sử dụng vector của người dùng hoặc vector đại diện cho phiên hiện tại để tìm các sản phẩm gần nhất trong không gian embedding.

5.2 Matrix Factorization bằng PyTorch

Matrix Factorization là một phương pháp nền tảng trong hệ thống gợi ý, đặc biệt phù hợp với dữ liệu tương tác người dùng-sản phẩm (user-item) có độ thưa cao [2]. Thay vì biểu diễn trực tiếp từng cặp người dùng-sản phẩm trong một ma trận lớn, phương pháp này học các vector ẩn có số chiều nhỏ hơn cho người dùng và sản phẩm. Quá trình huấn luyện điều chỉnh các vector này để những cặp có tương tác phù hợp nhận được điểm dự đoán cao hơn.

Trong mã nguồn của dự án, mô hình Matrix Factorization được định nghĩa bằng PyTorch. Mô hình gồm hai lớp embedding: một lớp cho người dùng và một lớp cho sản phẩm. Với mỗi cặp người dùng-sản phẩm, mô hình lấy vector tương ứng của người dùng và sản phẩm, sau đó tính tích vô hướng giữa hai vector này. Giá trị tích vô hướng được đưa qua hàm sigmoid để tạo ra điểm dự đoán trong khoảng từ 0 đến 1, biểu diễn khả năng người dùng quan tâm đến sản phẩm. Cơ chế tính điểm này được khái quát trong Hình 5.1.



Hình 5.1: Cách mô hình Matrix Factorization trong `src/recall_model/model.py` tính điểm cho một cặp người dùng-sản phẩm

Hình 5.1 cho thấy phần lõi của Recall là phép so khớp trong không gian embedding. Vì đầu ra của mô hình là điểm phù hợp cho từng cặp người dùng-sản phẩm, cùng một ma trận embedding sản phẩm có thể được tái sử dụng ở giai đoạn phục vụ để xây dựng chỉ mục FAISS và truy hồi ứng viên nhanh hơn.

Kích thước embedding trong cấu hình hiện tại là 64. Đây là lựa chọn thực nghiệm nhằm cân bằng giữa khả năng biểu diễn và chi phí tính toán. Nếu số chiều quá nhỏ, vector có thể không đủ khả năng biểu diễn sở thích đa dạng của người dùng và đặc trưng tiềm ẩn của sản phẩm. Ngược lại, nếu số chiều quá lớn, mô hình có thể tốn nhiều bộ nhớ hơn và dễ bị quá khớp.

5.3 Huấn luyện Recall và tìm kiếm vector bằng FAISS

Dữ liệu huấn luyện cho mô hình Recall được lấy từ tập `labeled_sessions.parquet`. Trước khi huấn luyện, hệ thống tách các mẫu tích cực và tiêu cực dựa trên nhãn đã tạo ở bước gán nhãn giả (*pseudo-labeling*). Vì dữ liệu thương mại điện tử có số lượng hành vi yếu và

không tích cực lớn hơn nhiều so với hành vi tích cực, dự án áp dụng lấy mẫu âm (negative sampling) để cân bằng dữ liệu huấn luyện. Cụ thể, với mỗi mẫu tích cực, hệ thống lấy tối đa bốn mẫu tiêu cực, tương ứng với tỷ lệ 1 mẫu tích cực và 4 mẫu tiêu cực.

Sau bước lấy mẫu, `user_id` và `product_id` được mã hóa thành các chỉ số số nguyên bằng `LabelEncoder`. Việc mã hóa này là cần thiết vì lớp embedding của PyTorch không làm việc trực tiếp với định danh gốc, mà cần các chỉ số liên tiếp từ 0 đến số lượng user hoặc item tương ứng. Các encoder này được lưu lại để sử dụng trong quá trình phục vụ suy luận.

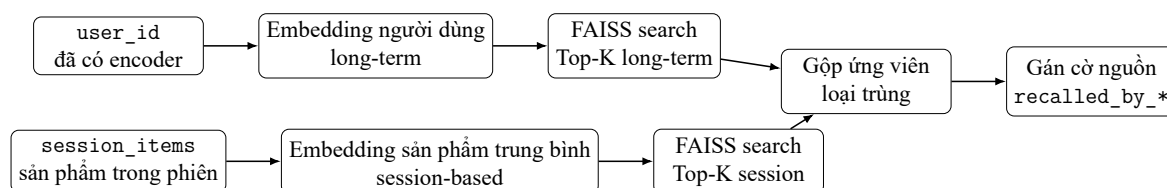
Mô hình được huấn luyện với kích thước batch là 4096, learning rate là 0.01 và số epoch là 5. Hàm mất mát được sử dụng là Focal Loss [9]. So với Binary Cross Entropy thông thường, Focal Loss giảm ảnh hưởng của các mẫu dễ và tập trung nhiều hơn vào các mẫu khó. Điều này phù hợp với dữ liệu phản hồi ẩn vì phần lớn tương tác có tín hiệu yếu, trong khi các hành vi có giá trị cao như thêm vào giỏ hàng hoặc mua hàng xuất hiện ít hơn.

Sau khi mô hình Recall được huấn luyện, vector embedding của sản phẩm được lưu thành file `item_embeddings.npy`. Đây là ma trận trong đó mỗi dòng tương ứng với vector biểu diễn của một sản phẩm. Để truy hồi sản phẩm nhanh, dự án sử dụng FAISS để xây dựng chỉ mục tìm kiếm vector từ ma trận embedding này [3]. Chỉ mục được sử dụng là `IndexFlatIP`, tức là tìm kiếm dựa trên tích vô hướng.

5.4 Dual-channel Recall

Một điểm đáng chú ý trong hệ thống là cơ chế Dual-Channel Recall, tức là truy hồi ứng viên từ hai nguồn tín hiệu khác nhau. Kênh thứ nhất là sở thích dài hạn (long-term preference), sử dụng user embedding để biểu diễn sở thích dài hạn của người dùng. Vector này được học từ lịch sử tương tác trước đó, do đó phù hợp với trường hợp người dùng quay lại hệ thống và chưa có nhiều hành vi trong phiên hiện tại.

Kênh thứ hai là sở thích theo phiên (session-based preference), sử dụng thông tin từ các sản phẩm người dùng đang tương tác trong phiên hiện tại. Nếu yêu cầu API có truyền tham số `session_items`, hệ thống sẽ ánh xạ các `product_id` này sang item index, lấy embedding tương ứng của từng sản phẩm và tính vector trung bình. Vector trung bình này được xem là biểu diễn cho nhu cầu tức thời của người dùng trong phiên. Sau đó, hệ thống sử dụng vector này để truy vấn FAISS và lấy ra các sản phẩm tương đồng với bối cảnh hiện tại. Quan hệ giữa hai kênh truy hồi được trình bày trong Hình 5.2.



Hình 5.2: Cơ chế Dual-Channel Recall được triển khai trong `src/serving/pipeline.py`

Hình 5.2 cho thấy hai kênh Recall bổ sung cho nhau thay vì thay thế nhau. Kênh long-term giúp hệ thống khai thác sở thích ổn định của người dùng trong quá khứ, trong khi kênh session-based giúp phản ánh nhu cầu đang thay đổi trong phiên hiện tại. Sau khi truy hồi từ hai kênh, hệ thống gộp danh sách ứng viên và loại bỏ sản phẩm trùng lặp. Mỗi ứng viên được gán thêm hai đặc trưng `recalled_by_long_term` và `recalled_by_session` để thể hiện nguồn gốc truy hồi.

5.5 Mô hình Ranking

Sau khi giai đoạn Recall tạo ra tập ứng viên, mô hình Ranking có nhiệm vụ chấm điểm chi tiết và sắp xếp lại các sản phẩm này. Nếu Recall tập trung vào tốc độ và khả năng bao phủ, Ranking tập trung vào chất lượng thứ tự của danh sách cuối cùng. Đây là giai đoạn quyết định sản phẩm nào sẽ được hiển thị ở các vị trí đầu tiên cho người dùng.

Dự án sử dụng LightGBM làm mô hình Ranking [4]. Về bản chất, mô hình được huấn luyện như một bộ chấm điểm nhị phân (binary scorer), dự đoán xác suất một ứng viên là sản phẩm có tương tác tích cực với người dùng. Tuy không phải là mô hình xếp hạng theo cặp hoặc theo danh sách đúng nghĩa, cách tiếp cận này vẫn phù hợp với phạm vi dự án vì LightGBM xử lý tốt dữ liệu dạng bảng, hỗ trợ trọng số mẫu và có tốc độ suy luận nhanh.

Đầu vào của LightGBM gồm nhiều nhóm đặc trưng được tạo từ pipeline dữ liệu của dự án. Nhóm đặc trưng người dùng phản ánh mức độ hoạt động và lịch sử tương tác của người dùng. Nhóm đặc trưng sản phẩm mô tả độ phổ biến, số lượng người dùng từng tương tác, giá trung bình và thông tin phân loại như thương hiệu hoặc danh mục. Nhóm đặc trưng phiên giúp mô hình nắm bắt ngữ cảnh ngắn hạn, còn nhóm đặc trưng nguồn Recall cho biết ứng viên được lấy ra từ kênh sở thích dài hạn, kênh phiên hiện tại hoặc cả hai.

5.6 Huấn luyện Ranking và Train-on-Recall

Dữ liệu huấn luyện Ranking được tạo bằng cách ghép dữ liệu tương tác đã gán nhãn với các bảng đặc trưng người dùng và sản phẩm. Trước khi huấn luyện, các cột phân loại như

category_code và brand được xử lý giá trị thiếu bằng nhãn <UNKNOWN> và chuyển sang kiểu category. Các cột số bị thiếu được điền bằng 0 để bảo đảm dữ liệu đầu vào không gây lỗi cho LightGBM.

Một điểm quan trọng trong quá trình huấn luyện là cách chia tập huấn luyện và tập kiểm thử. Thay vì chia ngẫu nhiên, dự án sắp xếp dữ liệu theo event_time và sử dụng cách chia theo thời gian (time-based split) với tỷ lệ 80% cho huấn luyện và 20% cho kiểm thử. Cách chia này mô phỏng tốt hơn bối cảnh triển khai, trong đó mô hình được học từ dữ liệu quá khứ và dự đoán cho các hành vi xảy ra sau đó.

Phiên bản Ranking cơ sở (baseline) được huấn luyện trực tiếp trên dữ liệu tương tác đã gán nhãn. Cách làm này đơn giản, nhưng có một hạn chế quan trọng: dữ liệu huấn luyện của Ranking có thể khác với dữ liệu mà mô hình gặp khi phục vụ suy luận. Trong thực tế, Ranking không chấm điểm toàn bộ dữ liệu tương tác gốc, mà chỉ chấm điểm các ứng viên do Recall sinh ra. Sự khác biệt này được gọi là lệch phân phối giữa huấn luyện và phục vụ (train-serving mismatch).

Để giảm vấn đề trên, dự án bổ sung hướng huấn luyện Train-on-Recall. Ý tưởng của phương pháp này là mô phỏng chính xác hơn pipeline phục vụ suy luận trong quá trình tạo dữ liệu huấn luyện cho Ranking. Cụ thể, hệ thống khởi tạo pipeline gợi ý, duyệt qua các phiên người dùng, sử dụng mô hình Recall và FAISS để sinh danh sách ứng viên giống như khi phục vụ API. Sau đó, các ứng viên này được gán nhãn dựa trên việc người dùng có thực sự tương tác với sản phẩm trong phiên hay không. Quy trình tạo tập huấn luyện này được minh họa trong Hình 5.3.



Hình 5.3: Quy trình tạo dữ liệu Train-on-Recall

Hình 5.3 cho thấy Train-on-Recall tạo ra tập huấn luyện Ranking gần với dữ liệu thực tế hơn. Các mẫu âm trong tập này không phải là các sản phẩm ngẫu nhiên hoàn toàn, mà là những sản phẩm đã được Recall xem là có tiềm năng nhưng người dùng không tương tác. Đây là các mẫu âm khó hơn và có giá trị huấn luyện cao hơn cho mô hình Ranking.

Chương 6

Triển khai API và phục vụ gợi ý

Chương này trình bày cách các mô hình và artifact đã huấn luyện được đưa vào luồng phục vụ trực tuyến. Nội dung bao gồm cách khởi động FastAPI server, các endpoint chính, quy trình xử lý request, cơ chế cold-start, xử lý thiếu đặc trưng và cách hệ thống ghi nhận độ trễ.

6.1 FastAPI server

Dự án sử dụng FastAPI để triển khai API phục vụ gợi ý sản phẩm [5]. FastAPI là một framework phù hợp cho các hệ thống suy luận mô hình vì có tốc độ tốt, cú pháp rõ ràng và hỗ trợ xây dựng REST endpoint một cách đơn giản. Trong dự án, file `main_serve.py` đóng vai trò là entrypoint cho phần phục vụ suy luận.

Khi server khởi động, ứng dụng tạo một đối tượng `RecommendationPipeline` và gọi phương thức `initialize()`. Quá trình này nạp các bảng kho đặc trưng, encoder, FAISS index, mô hình Recall PyTorch và mô hình Ranking LightGBM vào bộ nhớ. Việc nạp sẵn các thành phần này giúp API không phải đọc lại mô hình hoặc dữ liệu từ đĩa cho từng request, từ đó giảm đáng kể thời gian phản hồi.

Sau khi khởi động, API chạy tại địa chỉ mặc định `http://127.0.0.1:8000`. Người dùng hoặc ứng dụng client có thể gửi request đến các endpoint của hệ thống để lấy danh sách sản phẩm gợi ý.

6.2 Endpoint chính

Hệ thống cung cấp các endpoint chính phục vụ việc kiểm tra trạng thái và lấy danh sách gợi ý. Endpoint gốc / trả về thông báo đơn giản cho biết API đã sẵn sàng. Endpoint quan trọng nhất là `/recommend/{user_id}`, nhận vào định danh người dùng và trả về danh sách sản

phẩm gợi ý. Ngoài ra, hệ thống hỗ trợ tham số `session_items` để truyền thêm danh sách sản phẩm người dùng đang tương tác trong phiên hiện tại.

Với request `GET /recommend/{user_id}`, hệ thống chỉ sử dụng thông tin lịch sử dài hạn của người dùng để truy hồi ứng viên. Với request có thêm `session_items`, hệ thống kích hoạt thêm kênh Recall dựa trên phiên hiện tại. Endpoint `/recommend-by-index/{user_idx}` chủ yếu phục vụ kiểm thử nội bộ, trong đó người dùng được truy cập thông qua chỉ số đã mã hóa thay vì `user_id` gốc.

6.3 Quy trình xử lý yêu cầu

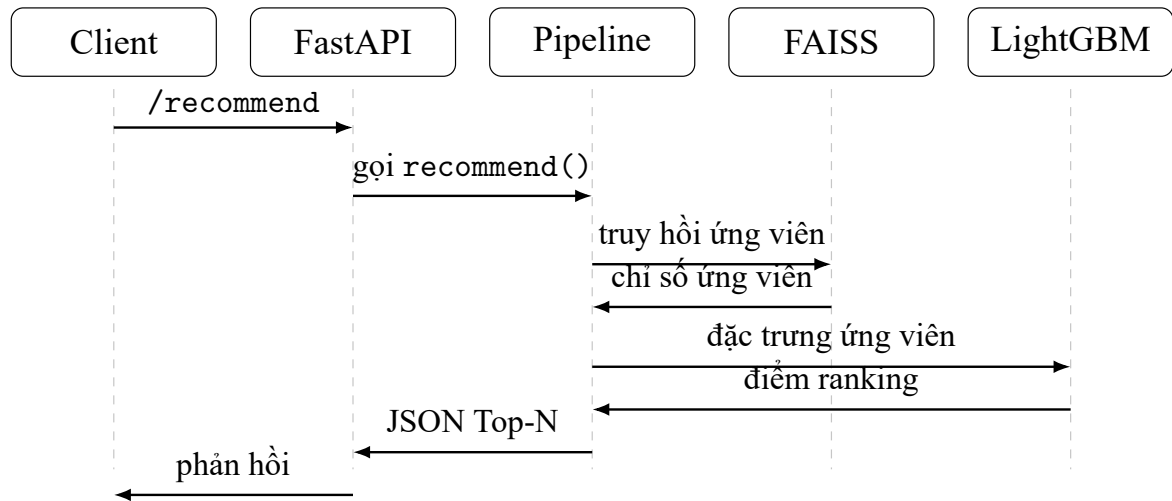
Khi nhận một request gợi ý, hệ thống trước tiên đọc `user_id`, tham số `top_n` và tùy chọn `session_items`. Nếu `session_items` được truyền qua tham số truy vấn (query parameter), chuỗi này sẽ được tách theo dấu phẩy và chuyển thành danh sách mã sản phẩm dạng số nguyên. Nếu định dạng không hợp lệ, API trả về lỗi để tránh đưa dữ liệu sai vào pipeline.

Sau khi nhận request hợp lệ, pipeline kiểm tra xem `user_id` có tồn tại trong encoder hay không. Nếu người dùng đã có trong dữ liệu huấn luyện, hệ thống ánh xạ `user_id` sang `user_idx` và lấy user embedding từ mô hình Recall. Vector này được sử dụng để truy vấn FAISS theo kênh dài hạn (long-term), tức là kênh biểu diễn sở thích dài hạn của người dùng.

Nếu request có truyền `session_items`, hệ thống tiếp tục xử lý kênh Recall dựa trên phiên hiện tại (session-based recall). Các `product_id` trong phiên được ánh xạ sang item index. Với những sản phẩm hợp lệ, pipeline lấy item embedding tương ứng từ mô hình Recall và tính vector trung bình. Vector này đại diện cho nhu cầu hiện tại của người dùng trong phiên. Sau đó, hệ thống truy vấn FAISS lần thứ hai để lấy các sản phẩm tương đồng với vector phiên.

Kết quả từ hai kênh Recall được gộp lại và loại bỏ trùng lặp. Với mỗi ứng viên, pipeline ghi nhận sản phẩm đó đến từ kênh long-term, kênh session-based hoặc cả hai. Thông tin này được chuyển thành hai đặc trưng `recalled_by_long_term` và `recalled_by_session`, giúp mô hình Ranking biết được nguồn gốc của từng ứng viên.

Sau khi có danh sách ứng viên, hệ thống tiến hành tra cứu đặc trưng. Đặc trưng sản phẩm được lấy từ `item_features`, còn đặc trưng người dùng được lấy từ `user_features`. Các đặc trưng động của phiên, chẳng hạn `user_session_interaction_count`, được tính trực tiếp tại thời điểm xử lý request dựa trên độ dài của danh sách `session_items`. Tập đặc trưng hoàn chỉnh sau đó được đưa vào LightGBM để dự đoán điểm phù hợp cho từng sản phẩm. Toàn bộ trao đổi giữa client, API, pipeline, FAISS và LightGBM được thể hiện trong Hình 6.1.



Hình 6.1: Luồng xử lý một yêu cầu gợi ý trong API FastAPI

Hình 6.1 cho thấy API chỉ đóng vai trò tiếp nhận và trả response, còn phần lớn logic gợi ý nằm trong RecommendationPipeline. Cách tổ chức này giúp tách lớp giao tiếp HTTP khỏi lớp suy luận mô hình, nhờ đó việc kiểm thử pipeline có thể thực hiện độc lập với FastAPI server.

6.4 Xử lý cold-start

Khởi đầu lạnh (cold-start) là vấn đề phổ biến trong hệ thống gợi ý. Trường hợp này xảy ra khi hệ thống gặp một người dùng mới chưa từng xuất hiện trong dữ liệu huấn luyện. Vì mô hình Matrix Factorization chỉ học embedding cho những người dùng đã có trong tập huấn luyện, hệ thống không thể lấy user embedding cho một user_id hoàn toàn mới.

Để tránh API bị lỗi trong tình huống này, pipeline triển khai cơ chế dự phòng (fallback) dựa trên sản phẩm phổ biến. Nếu user_id không tồn tại trong mapping, hệ thống bỏ qua giai đoạn Recall bằng user embedding và trả về danh sách các sản phẩm có item_total_interactions cao nhất. Đây là cách xử lý đơn giản nhưng hiệu quả trong phạm vi dự án, vì các sản phẩm phổ biến thường có xác suất được quan tâm cao hơn so với sản phẩm ít tương tác.

6.5 Xử lý thiếu đặc trưng khi phục vụ

Một rủi ro thường gặp khi triển khai mô hình học máy là sự không đồng bộ giữa đặc trưng lúc huấn luyện và đặc trưng lúc phục vụ. Nếu mô hình LightGBM được huấn luyện với một

danh sách cột nhất định nhưng dữ liệu phục vụ suy luận thiếu cột hoặc sai thứ tự cột, quá trình dự đoán có thể bị lỗi hoặc cho kết quả không chính xác.

Để giảm rủi ro này, lớp `LightGBMRanker` trong dự án tự động đọc danh sách đặc trưng mà mô hình yêu cầu thông qua `model.feature_name()`. Khi nhận dữ liệu ứng viên từ pipeline, lớp này kiểm tra xem tất cả các cột cần thiết đã tồn tại hay chưa. Nếu thiếu cột số, hệ thống bổ sung giá trị mặc định là 0.0. Nếu thiếu cột phân loại như `category_code` hoặc `brand`, hệ thống bổ sung giá trị `<UNKNOWN>`. Sau đó, dữ liệu đầu vào được sắp xếp lại đúng thứ tự đặc trưng mà mô hình `LightGBM` đã học.

6.6 Đo độ trễ

Đối với hệ thống gợi ý trực tuyến, độ trễ (latency) là một chỉ số quan trọng bên cạnh chất lượng mô hình. Người dùng thường kỳ vọng danh sách gợi ý được hiển thị gần như ngay lập tức, vì vậy pipeline phục vụ suy luận cần được thiết kế để hạn chế các thao tác tốn thời gian trong mỗi yêu cầu.

Trong dự án, phương thức `recommend_by_index()` ghi nhận thời gian xử lý của từng thành phần. Thời gian lấy user embedding phản ánh chi phí truy cập mô hình PyTorch để lấy vector người dùng. Thời gian tìm kiếm FAISS đo chi phí truy hồi ứng viên từ một hoặc hai kênh Recall. Thời gian tra cứu đặc trưng đo chi phí lấy đặc trưng từ các bảng kho đặc trưng đã nạp trong bộ nhớ. Thời gian xếp hạng phản ánh chi phí `LightGBM` dự đoán điểm cho toàn bộ ứng viên. Ngoài ra, hệ thống cũng đo thời gian sắp xếp kết quả và tổng độ trễ của request.

Chương 7

Kết quả thực nghiệm và đánh giá

Chương này tổng hợp các kết quả thực nghiệm sau khi chạy lại pipeline trong môi trường cục bộ của dự án, bao gồm kết quả của pipeline dữ liệu, mô hình Recall, mô hình Ranking và pipeline phục vụ suy luận. Các số liệu trong chương là kết quả ghi nhận từ quá trình chạy lệnh thực tế, không phải số liệu suy diễn từ hình minh họa. Khi cần tái lập kết quả trên một môi trường mới, cần chạy lại đầy đủ pipeline dữ liệu, huấn luyện Recall, huấn luyện Ranking và kiểm thử API theo các lệnh ở Phụ lục.

7.1 Kết quả pipeline dữ liệu

Sau khi chạy pipeline dữ liệu, dữ liệu thô được chuyển đổi thành các tập dữ liệu phục vụ huấn luyện và phục vụ suy luận. Pipeline thực hiện lần lượt các bước đọc dữ liệu, biến đổi giá sản phẩm, chia phiên người dùng, tạo snapshot đặc trưng, tạo đặc trưng theo thời điểm (point-in-time features) và gán nhãn cho dữ liệu tương tác. Kết quả đầu ra chính gồm `labeled_sessions.parquet`, `user_features.parquet` và `item_features.parquet`.

Tập `labeled_sessions.parquet` là dữ liệu huấn luyện đã được xử lý ở mức phiên-người dùng-sản phẩm. Điều này có nghĩa là nếu trong cùng một phiên, một người dùng tương tác nhiều lần với cùng một sản phẩm, hệ thống chỉ giữ lại dòng đại diện có tín hiệu mạnh nhất. Cách làm này giúp giảm trùng lặp và làm cho dữ liệu huấn luyện phản ánh rõ hơn mức độ quan tâm của người dùng đối với sản phẩm.

Hai tập `user_features.parquet` và `item_features.parquet` được sử dụng như kho đặc trưng dạng snapshot. `user_features.parquet` lưu các đặc trưng tổng hợp theo người dùng, chẳng hạn tổng số tương tác và tổng số phiên. `item_features.parquet` lưu các đặc trưng tổng hợp theo sản phẩm, bao gồm số lượt tương tác, số người dùng duy nhất, giá trung bình, thông tin danh mục, thương hiệu và độ phổ biến theo phiên.

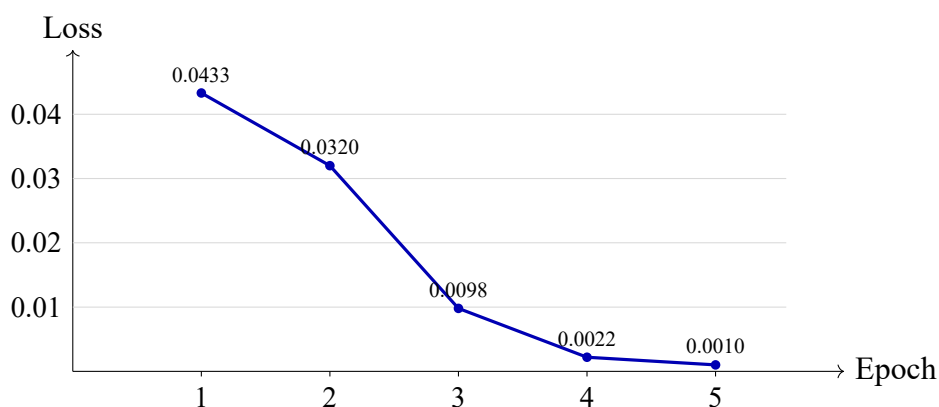
7.2 Kết quả Recall

Mô hình Recall được huấn luyện bằng Matrix Factorization với số chiều embedding là 64, kích thước batch là 4096, learning rate là 0.01 và số epoch là 5. Dữ liệu huấn luyện được cân bằng tương đối bằng lấy mẫu âm (negative sampling) với tỷ lệ 1 mẫu tích cực đi kèm tối đa 4 mẫu tiêu cực. Hàm mất mát được sử dụng là Focal Loss nhằm giảm ảnh hưởng của dữ liệu mất cân bằng.

Theo kết quả thực nghiệm được ghi nhận trong tài liệu của dự án, giá trị mất mát (loss) của mô hình Recall giảm nhanh qua các epoch. Ở epoch đầu tiên, loss còn ở mức 0.0433. Sau 5 epoch, loss giảm xuống 0.0010. Điều này cho thấy mô hình học được mối quan hệ giữa người dùng và sản phẩm trên tập dữ liệu huấn luyện. Bảng 7.1 trình bày các giá trị cụ thể, còn Hình 7.1 giúp quan sát xu hướng giảm loss trực quan hơn.

Bảng 7.1: Kết quả huấn luyện và đánh giá mô hình Recall

Chỉ số	Giá trị	Nhận xét
Loss epoch 1	0.0433	Mô hình bắt đầu học quan hệ người dùng-sản phẩm
Loss epoch 2	0.0320	Loss tiếp tục giảm
Loss epoch 3	0.0098	Mô hình hội tụ nhanh hơn
Loss epoch 4	0.0022	Loss tiến gần mức thấp
Loss epoch 5	0.0010	Loss tiếp tục giảm trên dữ liệu huấn luyện
Hit Rate@50	98.21%	Ghi nhận 603/614 tương tác tích cực trong đánh giá



Hình 7.1: Đường giảm loss của mô hình Recall qua 5 epoch

Hình 7.1 cho thấy loss giảm mạnh nhất trong ba epoch đầu và tiếp tục giảm ở hai epoch

cuối với biên độ nhỏ hơn. Kết quả Hit Rate@50 đạt 98.21%, tương ứng 603 trên 614 tương tác tích cực được tìm thấy trong Top-50 ứng viên khi chạy lại thực nghiệm trong môi trường cục bộ. Chỉ số này cho thấy mô hình Recall có khả năng bao phủ tốt các sản phẩm liên quan trong điều kiện đánh giá đã ghi nhận. Tuy nhiên, cần lưu ý rằng kết quả Recall rất cao cũng có thể phản ánh nguy cơ đánh giá lạc quan vì hàm `evaluate_recall()` trong `src/recall_model/run_recall.py` đang đánh giá trên các tương tác dương lấy từ cùng tập dữ liệu đã dùng để tạo vocabulary và huấn luyện, chưa phải một tập kiểm thử Recall tách biệt theo thời gian. Vì vậy, khi phát triển tiếp, nên đánh giá Recall trên tập kiểm thử tách theo thời gian để phản ánh chính xác hơn khả năng khái quát.

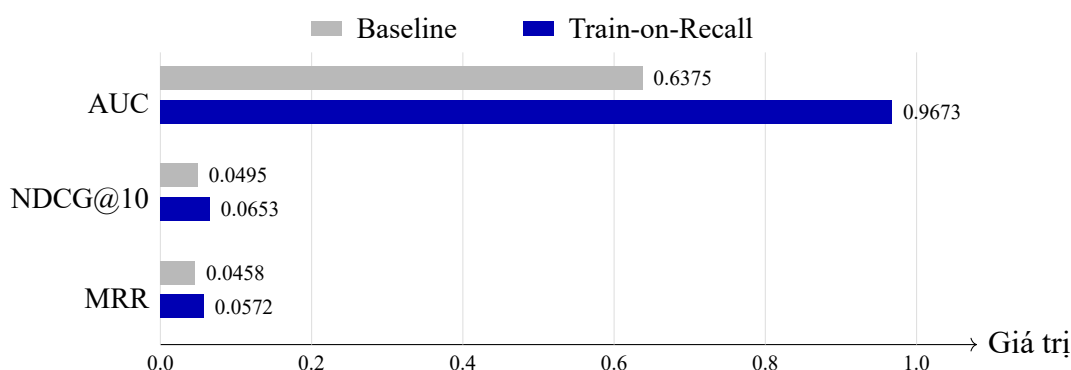
7.3 Kết quả Ranking

Mô hình Ranking được đánh giá bằng ba chỉ số chính: AUC, NDCG@10 và MRR. AUC phản ánh khả năng phân biệt giữa mẫu tích cực và mẫu không tích cực [10]. NDCG@10 đánh giá chất lượng thứ tự của 10 sản phẩm đầu tiên, trong đó các sản phẩm phù hợp xuất hiện ở vị trí cao sẽ có đóng góp lớn hơn [11]. MRR đo vị trí của sản phẩm phù hợp đầu tiên trong danh sách, từ đó phản ánh mức độ nhanh chóng mà người dùng có thể nhìn thấy sản phẩm liên quan [12].

Dự án có hai hướng huấn luyện Ranking: mô hình cơ sở (baseline) và mô hình Train-on-Recall. Mô hình baseline được huấn luyện trực tiếp trên dữ liệu tương tác đã gán nhãn, sau khi ghép với đặc trưng người dùng và đặc trưng sản phẩm. Mô hình Train-on-Recall được huấn luyện trên tập ứng viên sinh ra từ chính pipeline Recall, nhờ đó dữ liệu huấn luyện gần hơn với dữ liệu mà Ranking gặp khi phục vụ suy luận. Bảng 7.2 và Hình 7.2 được dùng để so sánh hai hướng huấn luyện này.

Bảng 7.2: Bảng so sánh hiệu năng xếp hạng giữa Baseline và Train-on-Recall

Mô hình	AUC	NDCG@10	MRR
Baseline	0.6375	0.0495	0.0458
Train-on-Recall	0.9673	0.0653	0.0572



Hình 7.2: So sánh AUC, NDCG@10 và MRR giữa Baseline và Train-on-Recall

Kết quả trong Hình 7.2 cho thấy Train-on-Recall cải thiện đáng kể so với baseline. AUC tăng từ 0.6375 lên 0.9673, cho thấy mô hình phân biệt tốt hơn giữa ứng viên có tương tác và ứng viên không có tương tác. NDCG@10 tăng từ 0.0495 lên 0.0653, còn MRR tăng từ 0.0458 lên 0.0572. Mức tăng của NDCG và MRR cho thấy thứ tự hiển thị của danh sách gợi ý đã được cải thiện, dù giá trị tuyệt đối vẫn chưa cao.

Việc NDCG@10 và MRR còn thấp là điều có thể giải thích được trong bối cảnh dữ liệu thương mại điện tử. Dữ liệu có độ thưa cao và tỷ lệ mẫu tích cực trong tập kiểm thử rất nhỏ. Khi nhiều nhóm người dùng hoặc phiên không có mẫu tích cực, các chỉ số xếp hạng trung bình sẽ bị kéo xuống. Do đó, các giá trị NDCG và MRR cần được đọc cùng với bối cảnh mất cân bằng dữ liệu, thay vì chỉ đánh giá theo độ lớn tuyệt đối.

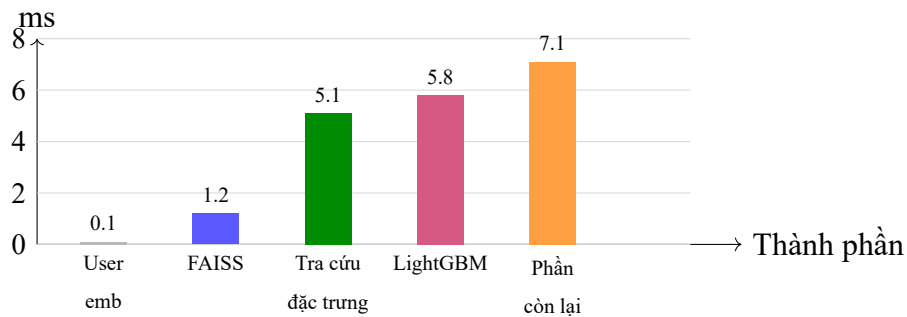
7.4 Kết quả phục vụ suy luận

Kết quả phục vụ suy luận được đánh giá thông qua thời gian khởi tạo pipeline và độ trễ khi xử lý request gợi ý. Khi API khởi động, hệ thống cần nạp kho đặc trưng, encoder, item embedding, FAISS index, trọng số mô hình Recall và mô hình LightGBM. Đây là chi phí xảy ra một lần khi server khởi động, không phải chi phí của từng request.

Đối với request gợi ý, hệ thống ghi nhận độ trễ của từng thành phần trong pipeline. Các thành phần chính gồm lấy user embedding, truy vấn FAISS, tra cứu kho đặc trưng, chấm điểm LightGBM, sắp xếp kết quả và tạo response. Khi chạy lại kiểm thử cục bộ với truy vấn hai kênh sau bước khởi động, độ trễ trung bình của 10 request đạt khoảng 19.3 ms. Bảng 7.3 và Hình 7.3 trình bày cách phân rã độ trễ theo từng thành phần.

Bảng 7.3: Bảng tổng hợp độ trễ tham chiếu của pipeline phục vụ

Thành phần	Thời gian	Tỷ trọng
Lấy user embedding	0.1 ms	0.5%
Tìm kiếm FAISS	1.2 ms	6.2%
Tra cứu đặc trưng	5.1 ms	26.4%
Xếp hạng LightGBM	5.8 ms	30.1%
Sắp xếp và phần còn lại	7.1 ms	36.8%
Tổng độ trễ	19.3 ms	100%



Hình 7.3: Biểu đồ phân rã độ trễ theo bảng tổng hợp pipeline phục vụ

Hình 7.3 cho thấy giai đoạn xếp hạng bằng LightGBM và tra cứu đặc trưng chiếm tỷ trọng đáng kể trong tổng thời gian xử lý vì mô hình phải chấm điểm nhiều ứng viên cùng lúc và ghép đặc trưng cho từng ứng viên. Trong lần chạy kiểm thử cục bộ, truy vấn hai kênh có `session_items` đạt trung bình khoảng 19.3 ms sau giai đoạn warm-up. Vì vậy, có thể kết luận rằng API đạt mục tiêu dưới 50 ms trong kịch bản thử nghiệm cục bộ đã ghi nhận, còn nút thắt hiệu năng cụ thể phụ thuộc vào dữ liệu, trạng thái bộ nhớ và loại yêu cầu.

7.5 Nhận xét và hạn chế

Từ các kết quả thực nghiệm, có thể thấy hệ thống đã xây dựng được một pipeline gợi ý tương đối hoàn chỉnh. Mô hình Recall giúp thu hẹp nhanh không gian tìm kiếm sản phẩm, trong khi FAISS bảo đảm quá trình truy hồi ứng viên diễn ra với độ trễ thấp. Mô hình Ranking giúp cải thiện chất lượng danh sách cuối cùng bằng cách sắp xếp lại các ứng viên theo nhiều nhóm đặc trưng khác nhau.

Dù vậy, hệ thống vẫn còn một số hạn chế. Kho đặc trưng hiện tại là các file Parquet được nạp vào RAM, phù hợp với môi trường thực nghiệm nhưng chưa phải là giải pháp tối ưu cho

hệ thống lớn hoặc dữ liệu thay đổi liên tục. Hệ thống cũng chưa có cơ chế cập nhật dữ liệu theo thời gian thực. Ngoài ra, đánh giá Recall cần được tách huấn luyện/kiểm thử chặt chẽ hơn để tránh kết quả quá lạc quan, còn mô hình Ranking hiện vẫn dùng binary objective thay vì tối ưu trực tiếp cho mục tiêu xếp hạng.

Chương 8

Kết luận và hướng phát triển

Chương cuối cùng tổng kết những nội dung đã thực hiện trong dự án và chỉ ra các hướng cải tiến có thể triển khai trong tương lai. Phần kết luận tập trung vào mức độ hoàn thiện của hệ thống hiện tại, còn phần hướng phát triển nêu các cải tiến về thuật toán, đặc trưng và hạ tầng phục vụ.

8.1 Kết luận

Trong bài tập lớn này, đề tài đã xây dựng một hệ thống gợi ý sản phẩm thương mại điện tử theo kiến trúc hai giai đoạn. Hệ thống bắt đầu từ dữ liệu hành vi người dùng, thực hiện các bước tiền xử lý, chia phiên, gán nhãn, tạo đặc trưng, huấn luyện mô hình và cuối cùng triển khai API để trả về danh sách sản phẩm gợi ý. Qua đó, dự án thể hiện được một luồng xử lý tương đối đầy đủ của một hệ thống gợi ý, từ dữ liệu thô đến kết quả phục vụ người dùng.

Ở giai đoạn xử lý dữ liệu, hệ thống đã chuyển đổi dữ liệu hành vi thành tập dữ liệu có cấu trúc phù hợp cho học máy. Các hành vi như xem sản phẩm, thêm vào giỏ hàng và mua hàng được ánh xạ thành nhãn và trọng số mẫu, giúp mô hình phân biệt được mức độ quan trọng của từng loại tương tác. Các đặc trưng người dùng, sản phẩm và phiên được xây dựng theo hướng point-in-time nhằm hạn chế rò rỉ dữ liệu tương lai và làm cho quá trình huấn luyện gần hơn với điều kiện thực tế.

Ở giai đoạn mô hình, hệ thống sử dụng Matrix Factorization bằng PyTorch để học biểu diễn vector cho người dùng và sản phẩm. Các item embedding sau huấn luyện được đưa vào FAISS để hỗ trợ truy hồi ứng viên nhanh. Trên tập ứng viên này, mô hình LightGBM được sử dụng để chấm điểm và xếp hạng lại sản phẩm dựa trên nhiều nhóm đặc trưng khác nhau. Cách kết hợp Recall và Ranking giúp hệ thống cân bằng giữa tốc độ truy hồi và chất lượng danh sách gợi ý.

Ở giai đoạn phục vụ, dự án triển khai API bằng FastAPI. Pipeline phục vụ suy luận nạp sẵn kho đặc trưng, mô hình Recall, FAISS index và mô hình Ranking vào bộ nhớ khi server khởi động. Khi nhận request, hệ thống có thể gợi ý theo sở thích dài hạn của người dùng và bổ sung thêm thông tin phiên hiện tại thông qua cơ chế Dual-Channel Recall. Ngoài ra, hệ thống cũng có cơ chế dự phòng cho người dùng mới và cơ chế tự căn chỉnh đặc trưng đầu vào cho LightGBM, giúp API ổn định hơn trong quá trình suy luận.

Theo các kết quả thực nghiệm được ghi nhận trong tài liệu của dự án, hướng tiếp cận hai giai đoạn là phù hợp với bài toán gợi ý sản phẩm. Mô hình Recall có khả năng lọc nhanh các sản phẩm ứng viên, mô hình Ranking cải thiện thứ tự hiển thị, còn API đạt độ trễ dưới 50 ms trong các kịch bản thử nghiệm cục bộ đã ghi nhận. Dù vậy, hệ thống vẫn còn một số hạn chế liên quan đến kho đặc trưng, cập nhật dữ liệu thời gian thực và phương pháp đánh giá. Đây là cơ sở để tiếp tục cải tiến trong các phiên bản sau.

8.2 Hướng phát triển

Trong các hướng phát triển tiếp theo, mô hình Recall có thể được cải tiến bằng cách sử dụng BPR Loss thay cho cách học pointwise hiện tại [8]. BPR Loss phù hợp hơn với bản chất của bài toán gợi ý vì trực tiếp tối ưu quan hệ thứ tự giữa sản phẩm tích cực và sản phẩm tiêu cực của cùng một người dùng. Việc áp dụng phương pháp này có thể giúp tập ứng viên Recall có chất lượng tốt hơn.

Đối với mô hình Ranking, hệ thống có thể chuyển từ binary objective sang các objective tối ưu trực tiếp cho xếp hạng như LambdaRank hoặc LambdaMART [13]. Những phương pháp này được thiết kế để cải thiện các chỉ số như NDCG, do đó phù hợp hơn với mục tiêu đưa sản phẩm liên quan lên các vị trí đầu danh sách. Ngoài ra, mô hình Ranking cũng có thể được bổ sung các đặc trưng tương tác chéo như mức độ yêu thích của người dùng với từng danh mục, thương hiệu hoặc phân khúc giá.

Về mặt hạ tầng, kho đặc trưng hiện tại mới được triển khai đơn giản bằng các file Parquet nạp vào RAM [7]. Khi dữ liệu lớn hơn hoặc cần cập nhật thường xuyên hơn, hệ thống nên chuyển sang một giải pháp feature store chuyên dụng, chẳng hạn Feast, để đồng bộ tốt hơn giữa đặc trưng offline dùng cho huấn luyện và đặc trưng online dùng cho phục vụ suy luận [14]. Bên cạnh đó, FAISS index hiện sử dụng IndexFlatIP, phù hợp với quy mô thử nghiệm nhưng có thể cần thay bằng HNSW hoặc IVF khi số lượng sản phẩm tăng lên rất lớn.

Cuối cùng, hệ thống cần được đánh giá trong môi trường gần thực tế hơn. Ngoài các chỉ số offline như AUC, NDCG và MRR, nên bổ sung đánh giá trực tuyến (online evaluation) hoặc A/B testing để đo lường tác động của hệ thống đối với hành vi người dùng thật. Đây là

bước cần thiết nếu muốn đưa hệ thống từ mức bài tập lớn hoặc thử nghiệm cục bộ sang môi trường sản phẩm thực tế.

Tài liệu tham khảo

- [1] Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*, pages 191–198, 2016.
- [2] Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.
- [3] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.
- [4] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Wipeng Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in neural information processing systems*, pages 3146–3154, 2017.
- [5] FastAPI. Fastapi documentation. <https://fastapi.tiangolo.com/>, 2026. Truy cập ngày 28/05/2026.
- [6] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [7] Apache Software Foundation. Apache parquet documentation. <https://parquet.apache.org/docs/>, 2026. Truy cập ngày 28/05/2026.
- [8] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. Bpr: Bayesian personalized ranking from implicit feedback. In *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*, pages 452–461. AUAI Press, 2009.

- [9] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollar. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2980–2988, 2017.
- [10] Tom Fawcett. An introduction to roc analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006.
- [11] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of ir techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.
- [12] Nick Craswell. Mean reciprocal rank. In *Encyclopedia of Database Systems*, page 1703. Springer, 2009.
- [13] Christopher J. C. Burges. From ranknet to lambdarank to lambdamart: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010.
- [14] Feast Project. Feast documentation. <https://docs.feast.dev/>, 2026. Truy cập ngày 28/05/2026.

Phụ lục A

Phụ lục

Phụ lục cung cấp các thông tin hỗ trợ để người đọc có thể kiểm tra và chạy lại các thành phần chính của dự án. Các nội dung bao gồm lệnh chạy chương trình, ví dụ yêu cầu API, cấu trúc phản hồi JSON và bảng siêu tham số được sử dụng trong hệ thống.

A.1 Lệnh chạy chương trình

Các lệnh dưới đây được sử dụng để cài đặt môi trường, chạy pipeline huấn luyện và khởi động API phục vụ gợi ý:

```
uv sync
uv run python main_train.py
uv run python -m src.ranking_model.train_on_recall
uv run python main_serve.py
```

A.2 Ví dụ yêu cầu API

Sau khi FastAPI server được khởi động, có thể kiểm tra endpoint gợi ý bằng các request sau:

```
curl "http://127.0.0.1:8000/recommend/10005"
curl "http://127.0.0.1:8000/recommend/10005?\
session_items=5700030,5700140"
```

A.3 Ví dụ phản hồi JSON

Đoạn dưới đây chỉ minh họa cấu trúc phản hồi JSON của API, không phải kết quả đo thực nghiệm hay một phản hồi được ghi nhận cố định từ hệ thống:

```
{
  "user_id": 10005,
  "user_idx": 10,
  "recommendations": [
    {
      "product_id": 5700030,
      "item_idx": 123,
      "score": 0.8123
    }
  ],
  "latency_metrics": {
    "user_emb_time_ms": 0.0,
    "faiss_search_time_ms": 0.0,
    "feature_lookup_time_ms": 0.0,
    "ranking_time_ms": 0.0,
    "sort_time_ms": 0.0,
    "total_latency_ms": 0.0
  }
}
```

A.4 Bảng siêu tham số

Bảng A.1: Một số tham số cấu hình chính của hệ thống

Nhóm	Tham số	Giá trị
Pipeline dữ liệu	Ngưỡng chia phiên	30 phút
Recall	Số chiều embedding	64
Recall	Kích thước batch	4096
Recall	Số epoch	5
Recall	Learning rate	0.01
Recall	Tỷ lệ lấy mẫu âm	1:4
Ranking	Learning rate	0.05
Ranking	Số lá	31
Ranking	Số vòng boosting	100
Phục vụ	Top-K long-term recall	100
Phục vụ	Top-K session recall	100
Phục vụ	Top-N ranking	20